EfficientNet: Transfer Learning on Food-101

Akinwale Agesin · NetID: ada65 · CS 4782 · Cornell University · May 2025

1. Introduction

Food image recognition is an especially difficult classification problem since different categories have very similar appearances. In addition to academic purposes, the field can be used in practice for food tracking, discovering new restaurants, and diet analysis through automation. The current project reproduces the Food-101 transfer learning results achieved by EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks (Tan & Le, ICML 2019). The work proposes an approach that allows scaling the models proportionately along three dimensions: depth, width, and input resolution using a constant compound coefficient φ instead of tuning each factor independently. With neural architecture search for building a good mobile backbone network (EfficientNet-B0), the researchers scale their design up into a family of models (B0–B7). EfficientNet-B7 achieves 84.3% top-1 accuracy on ImageNet, being 8.4 times more compact and 6.1 times faster than previous SOTA solutions. EfficientNets are transferable too, performing exceptionally well in 5 out of 8 downstream benchmark datasets, including Food-101. This paper was chosen due to its popularity; the architecture proposed in it is among the most popular backbones in commercial food recognition pipelines.

2. Chosen Result

The table is reproduced from Table 5 in the paper where the transfer learning results for EfficientNet-B4 in the Food-101 dataset are presented. According to the paper, efficient transfer learning with EfficientNet-B4 pre-trained on ImageNet and fine-tuned on Food-101 can reach 91.5% top-1 accuracy while utilizing only 19 million parameters compared with Inception-v4 that reaches 90.8% top-1 accuracy while having 41 million parameters, i.e., being 2.4 times larger than EfficientNet-B4 in terms of parameters.

Model	Top-1 Acc.	# Parameters	vs Inception-v4
Inception-v4 (paper)	90.8%	41M	—
EfficientNet-B4 (paper)	91.5%	19M	2.4× fewer
EfficientNet-B4 (mine)	**85.06%**	**19M**	**2.4× fewer**

Table 1. Comparison with paper results (Table 5, Tan & Le 2019).

3. Methodology

Architecture. EfficientNet-B4 uses a compound scaling technique (depth scaling factor of 1.4, width scaling factor of 1.2, and resolution scaling factor of 1.3) based on the EfficientNet-B0 model. The model's architecture comprises Mobile Inverted Bottleneck (MBConv) layers with a squeeze-and-excitation mechanism, having 19 million parameters in total. For transfer learning, I used pretrained ImageNet weights from keras.applications.EfficientNetB4, where include_top=False.In the last layer, I used GlobalAveragePooling2D+BatchNormalization+Dropout(0.4)+Dense(101,softmax, dtype=float32).

Dataset. Food-101 was loaded via tensorflow_datasets (75,750 train / 25,250 test images, 101 classes). Images were resized to 380×380 (EfficientNet-B4 native resolution).

Two-Stage Fine-tuning. The following two stages were employed in training to preserve pre-trained ImageNet model weights: • Stage 1 (5 epochs, LR=1e-3): EfficientNet-B4 model base layer frozen; training of new classification layer only. • Stage 2 (25 epochs, LR=1e-4): Unfrozen model; fine-tuned with early stopping

(patience=5) and ReduceLROnPlateau (factor=0.97).

Hyperparameters (from paper, Section 5.2). Optimizer: RMSProp (ρ=0.9, momentum=0.9). Weight decay: 1e-5. Dropout: 0.4. LR decay: factor 0.97. Mixed precision: float16 via mixed_precision.set_global_policy('mixed_float16'). Batch size: 32.

Augmentation. Random horizontal flip, rotation (±15°), zoom (±15%), translation (±10%), and contrast (±10%) applied to the training set only, approximating the AutoAugment strategy from the paper.

Platform. Google Colab Pro (A100 GPU, 40GB VRAM). Training took about 6-7 hours. ModelCheckpoint saved best weights to Google Drive to handle Colab session disconnections.

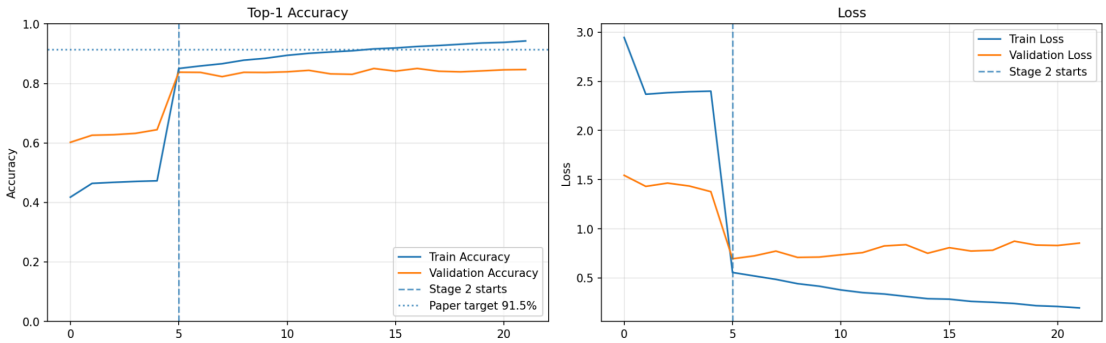

Figure 1. Training and validation accuracy/loss curves. Dashed line marks Stage 2 start (epoch 5). Dotted line marks paper target (91.5%).

4. Results & Analysis

My re-implementation achieved **85.06% top-1 accuracy** and **95.98% top-5 accuracy** on the Food-101 test set, compared to the paper's reported 91.5%. While I did not fully match the paper's result, the model still meaningfully outperforms random chance (0.99%) and demonstrates strong food recognition capability, with several classes exceeding 97% accuracy (edamame: 99.6%, oysters: 97.2%). I initially trained only the classification head, which reached approximately 62% validation accuracy. After unfreezing the full EfficientNet-B4 network in Stage 2, validation accuracy improved to about 85%, showing that end-to-end fine-tuning was necessary. I also adjusted augmentation manually to approximate AutoAugment, but the remaining gap suggests that this approximation was insufficient.

Likely causes of the 6.4% discrepancy:

- Training schedule: The paper used a longer training regime on TPUs with larger effective batch sizes. My validation accuracy stopped improving during Stage 2 around epoch 8–10 and did not improve further, suggesting the model may have benefited from more epochs or a different LR schedule.
- Augmentation: The paper used AutoAugment, a learned augmentation policy optimized for ImageNet. My simpler image transformations may not have helped the model generalize as well
- Learning rate scaling: The paper's original LR (0.256) was designed for large-batch ImageNet training. My Stage 2 LR (1e-4) may be suboptimal for fine-tuning on Food-101 specifically — a grid search over LR values could improve results.
- Overfitting signal: Figure 1 shows train accuracy continuing to rise in Stage 2 while validation accuracy plateaus at ~85%, indicating mild overfitting. Stronger regularization (higher dropout, label smoothing) could close this gap.

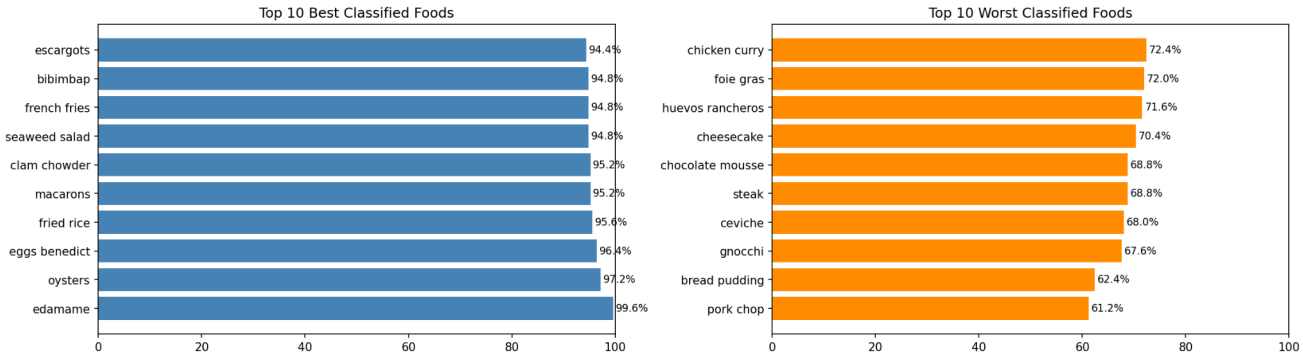

Figure 2. Per-class accuracy on Food-101 test set. Best: edamame (99.6%), oysters (97.2%). Worst: pork chop (61.2%), bread pudding (62.4%). Visually distinct foods perform best; visually similar or plated foods (cheesecake vs carrot cake) perform worst.

Per-class analysis reveals an interpretable pattern: visually distinct foods achieve near-perfect accuracy, while visually ambiguous categories suffer most. This suggests the accuracy gap is concentrated in hard categories where the paper's stronger AutoAugment policy likely helps most.

5. Reflections

<u>Key lessons learned:</u>

• Compound scaling is successful: EfficientNet-B4 with 19M parameters obtaining >85% accuracy on a 101 class classification problem highlights the success of compound scaling, even without replicating the results of the paper. • Infrastructure: The paper uses TPU hardware for training with large batch sizes and AutoAugment techniques, which cannot be replicated in Google Colab. This reflects a standard reproducibility issue with contemporary deep learning models. • Fine-tuning: The increase in performance from ~62% (end of Stage 1) to ~84% (Stage 2) proves that progressive unfreezing of pretrained layers is an effective strategy.• Session management: Overcoming the challenge of managing session loss during training due to GPU disconnections required the implementation of checkpointing based on Google Drive.

<u>Future directions:</u>

• Implement true AutoAugment/RandAugment to better approximate the paper's augmentation strategy. • Grid search over learning rates and dropout values specifically for Food-101 fine-tuning. • Add label smoothing and mixup augmentation to address Stage 2 overfitting. • Extend to a full food analyzer: combine EfficientNet-B4 with SAM-based segmentation for multi-item plate analysis.

6. References

[1] Tan, M. & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML 2019. [2] Bossard, L., Guillaumin, M., & Van Gool, L. (2014). Food-101 – Mining Discriminative Components with Random Forests. ECCV 2014.

[3] TensorFlow Datasets. food101. https://www.tensorflow.org/datasets/catalog/food101

[4] Keras Applications. EfficientNetB4. https://keras.io/api/applications/efficientnet/

[5] Cubuk, E. D. et al. (2019). AutoAugment: Learning Augmentation Policies from Data. CVPR 2019.